

- ✓ Dashboards
- ✓ Metrics
- ✓ Logs
- ✓ Traces
- ✓ Alerting
- ✓ Kubernetes
- ✓ Production
- ✓ Real-World Examples

Learn
Build
Monitor
Operate



Grafana

IN PRODUCTION

A PRACTICAL GUIDE FOR JUNIOR ENGINEERS



METRICS



LOGS



TRACES



ALERTS



DASHBOARDS



REAL-WORLD
PRACTICE

From Setup
to Real-World
Monitoring

- ✓ Step-by-Step Learning
- ✓ Practical Examples
- ✓ Production Best Practices
- ✓ Troubleshooting Scenarios
- ✓ Final Project

VERIQTA

TECH EDUCATION FOR A BRIGHTER TOMORROW

Build
Your Future

Grafana

in the Production Observability Stack

TURNING TELEMETRY INTO INSIGHTS

WHAT IS GRAFANA?

- Grafana is an open source platform used to visualize, explore and analyze data from multiple sources.
- It helps you create dashboards, set up alerts and understand the health of your systems in real time.
- Engineers use Grafana to turn metrics, logs and traces into meaningful insights.

WHAT GRAFANA IS NOT

- ✗ It is not a database.
- ✗ It does not store your metrics, logs or traces (long term).
- ✗ It is not a monitoring tool by itself.
- ✗ It does not collect data from your servers or applications.
- ✗ It will not make your system observable just by installing it.

VISUALIZATION vs TELEMETRY STORAGE

Grafana (Visualization)

- Queries data from sources
- Visualizes data in dashboards
- Helps with alerting
- Provides a single view

Telemetry Storage (Examples)

- Prometheus stores metrics
- Loki stores logs
- Tempo stores traces
- Other sources: Elasticsearch, CloudWatch, InfluxDB, etc.

METRICS, LOGS AND TRACES



METRICS

- Numerical data over time
- Used for monitoring and alerting
- Examples: CPU, memory, request rate, latency



LOGS

- Detailed events and messages
- Used for troubleshooting
- Examples: error logs, access logs, application logs



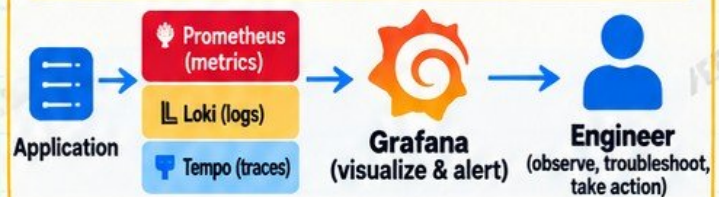
TRACES

- Show the path of a request across services
- Used for performance analysis
- Examples: request flow, service latency, bottlenecks

GRAFANA'S ROLE IN OBSERVABILITY

- ✓ Provides a unified view of metrics, logs and traces.
- ✓ Helps engineers detect and investigate issues.
- ✓ Enables alerting and notifications.
- ✓ Supports multiple data sources.
- ✓ Makes complex system data easy to understand.
- ✓ Used by DevOps, SRE and development teams.

BASIC PRODUCTION FLOW



WHY DASHBOARDS ALONE DO NOT CREATE OBSERVABILITY

- ✗ A dashboard only shows data, it does not collect or store it.
- ✗ You can have beautiful dashboards and still miss critical issues.
- ✗ Observability also requires proper instrumentation, logging, tracing, alerting, and good system design.
- ✗ Dashboards are a tool, not a strategy. Observability is a combination of people, processes and the right telemetry.



JUNIOR ENGINEER TIP

Grafana usually queries data stored elsewhere, such as Prometheus for metrics, Loki for logs and Tempo for traces. Always understand where the data comes from.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



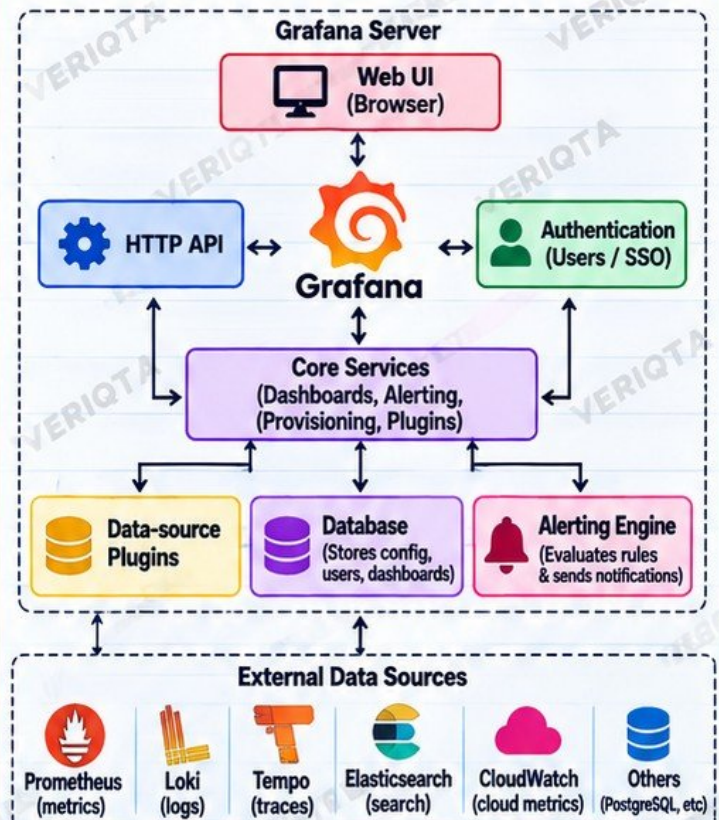
Grafana Architecture

Understanding the main components and how they work together

KEY COMPONENTS

- 1 **Grafana Server** Core application that runs Grafana.
- 2 **Web UI** Browser interface for users to create dashboards, explore data and manage settings.
- 3 **HTTP API** Used for automation, integration and managing resources.
- 4 **Database** Stores Grafana configuration, users, dashboards and metadata.
- 5 **Data-source plugins** Allow Grafana to connect to different data sources (Prometheus, Loki, Tempo, etc).
- 6 **Authentication** Manages users, teams and access (roles, SSO, OAuth, etc).
- 7 **Provisioning** Automates configuration of dashboards, data sources and alerting via files or API.
- 8 **Alerting engine** Evaluates alert rules and sends notifications.
- 9 **Plugin system** Extends Grafana with additional panels, data sources and features.

GRAFANA ARCHITECTURE DIAGRAM



REQUEST FLOW



WHAT HAPPENS WHEN A DASHBOARD LOADS?

- 1 You open a dashboard in the browser.
- 2 Grafana loads the dashboard configuration from its database.
- 3 Grafana sends a query to the relevant data source (e.g. Prometheus).
- 4 The data source returns the data.
- 5 Grafana processes the data and renders it in panels.
- 6 Panels are displayed in your browser.

IMPORTANT NOTES

- ✓ Grafana does not store your metrics, logs or traces (long term).
- ✓ It queries data from external sources.
- ✓ Multiple data sources can be used in one dashboard.
- ✓ The database stores Grafana configuration, not large amounts of telemetry data.
- ✓ Grafana can be extended with plugins.
- ✓ It can be fully automated using provisioning and APIs.

JUNIOR ENGINEER TIP



Grafana usually queries data stored elsewhere. It is a visualization and analysis tool, not a telemetry storage system.

Good
Observability
Builds Reliable
Systems



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Building a Production Grafana Environment

Different ways to deploy Grafana and what you need to run it in production

1. LOCAL INSTALLATION vs PRODUCTION

Local Installation

- Used for learning and testing
- Quick and simple setup
- Usually single instance
- Not highly available
- Data may be lost if not configured with persistence

Production Deployment

- Used in real environments
- More secure and scalable
- Configured with persistent storage
- Integrated with authentication and monitoring
- Deployed using Linux, Docker, Kubernetes or managed service

2. DEPLOYMENT OPTIONS



Linux Package

- Install using package manager (apt/yum)
- Common for self-managed servers



Docker

- Quick and consistent deployment
- Good for testing and small environments



Kubernetes

- Best for production and large scale
- Use Helm chart (grafana/grafana)



Managed Grafana

- Offered by cloud providers (AWS, Azure, GCP)
- Reduced operational overhead
- Good for teams that don't want to manage infrastructure

3. PERSISTENT STORAGE



- Stores Grafana configuration, users, dashboards and plugins
- Use a persistent volume (Docker/K8s)
- Common options: local disk, NFS, EBS
- Without persistence, all data is lost when the container is deleted

4. CONFIGURATION FILES



- Main configuration file: [grafana.ini](#)
- Controls server settings, database, security, logging
- Location (Linux): [/etc/grafana/grafana.ini](#)
- Can also use environment variables to override settings

5. ENVIRONMENT VARIABLES



- Used to configure Grafana without editing files
- Example: [GF_SERVER_ROOT_URL](#), [GF_DATABASE_TYPE](#)
- Useful in Docker and Kubernetes
- Makes configuration easier to manage and automate

6. PORTS AND NETWORKING



- Default port: [3000](#) (HTTP)
- Can be changed in [grafana.ini](#)
- Ensure the port is open in your firewall/security group
- When using a reverse proxy (Nginx), use a domain and HTTPS

7. WHY A DISPOSABLE CONTAINER IS DANGEROUS



- When the container is deleted:
 - All dashboards, users and settings are lost
 - You have to start from scratch
 - Not suitable for production
 - Always use persistent storage for real environments

JUNIOR ENGINEER TIP



Use Docker for learning, but always configure persistent storage when moving to production. Grafana stores important data, so never rely on a disposable container.

8. PRACTICE: START GRAFANA LOCALLY



1 Run Grafana using Docker

```
docker run -d --name grafana -p 3000:3000 \\\n-v grafana-data:/var/lib/grafana grafana/grafana
```

2 Open in browser

<http://localhost:3000>

3 Verify health

```
docker ps \ncurl http://localhost:3000/api/health
```

4 Login (default credentials)

Username: admin
Password: admin

Observability
Starts with a
Strong Foundation



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Grafana Configuration for Production

Configure Grafana correctly for a secure, stable and maintainable production environment

1. grafana.ini

- Main configuration file for Grafana.
- Controls server, database, security, logging and many other settings.
- Located at: `/etc/grafana/grafana.ini`
- Organized into sections.

```
[server]
root_url = https://grafana.example.com
serve_from_sub_path = false

[database]
type = sqlite3
path = /var/lib/grafana/grafana.db

[security]
admin_user = admin
admin_password = strongpassword
```

2. Environment-Variable Overrides

- You can override `grafana.ini` settings using environment variables.
- Useful in Docker, Kubernetes and cloud environments.
- Format: `GF_SECTION_KEY=value`

```
# Examples
GF_SERVER_ROOT_URL=https://grafana.example.com
GF_DATABASE_TYPE=postgres
GF_SECURITY_ADMIN_USER=admin
GF_SECURITY_ADMIN_PASSWORD=secret
GF_LOG_LEVEL=info
```

3. Server Settings

- Controls how Grafana runs and listens for requests.
- Key settings:

```
http_port = 3000
domain = grafana.example.com
root_url = https://grafana.example.com
serve_from_sub_path = false
```

Note:

- `root_url` must match the URL users access (important for alerts and links).
- If running behind a reverse proxy (Nginx), configure the correct domain and protocol.

4. Domain and Root URL

- `domain`: the domain name of your Grafana.
- `root_url`: the full URL including protocol.
- Used for:
 - Generating links in alerts
 - OAuth/SSO callbacks
 - Sharing dashboards
 - Running behind a reverse proxy

Example:

```
domain = grafana.example.com
root_url = https://grafana.example.com
```

5. Database Configuration

- Stores Grafana configuration, users, dashboards and metadata.
- Supported databases:
 - SQLite (default, for testing)
 - PostgreSQL (recommended)
 - MySQL
 - MSSQL

```
[database]
type = postgres
host = localhost:5432
name = grafana
user = grafana
password = strongpassword
ssl_mode = disable
```

6. Logging

- Controls log level and log output.
- Helps with troubleshooting.
- Key settings:

```
[log]
level = info
mode = console
filters = rendering:debug
```

Use `level = debug` temporarily when investigating issues, then change back to reduce noise.

7. Security Settings

- Controls authentication, access and security features.
- Key settings:

```
[security]
admin_user = admin
admin_password = strongpassword
disable_gravatar = true
cookie_secure = true
cookie_samesite = lax
```

8. Feature Configuration

- Enable or disable Grafana features.
- Examples:
 - Alerting
 - Anonymous access
 - Public dashboards
 - Explore
 - Plugins

```
[feature_toggles]
enable = alerting,explore
[auth.anonymous]
enabled = false
```

9. Restart Requirements

- Most configuration changes require a Grafana restart to take effect.
- Restart the service after editing `grafana.ini` or changing environment variables.

```
# Linux
sudo systemctl restart grafana-server

# Docker
docker restart grafana

# Kubernetes (if using a Deployment)
kubectl rollout restart deployment grafana
```

10. Configuration Precedence

Order of precedence (highest to lowest):

- 1 Environment variables (`GF_*`)
- 2 Command-line arguments
- 3 `grafana.ini` file
- 4 Default values



Production Awareness

- ✓ Configuration should be reproducible.
- ✓ Use version control (Git) for `grafana.ini` and provisioning files.
- ✓ Use environment variables for environment-specific settings.
- ✓ Document your configuration.
- ✓ Test changes in a non-production environment first.



Key Takeaway



A well-configured Grafana is secure, stable, and easy to maintain. Always manage your configuration as code.

Connecting Production Data Sources

Integrate real data sources with Grafana to visualize, explore and monitor your systems

1. WHAT IS A DATA SOURCE?



- A data source is a connection between Grafana and a system that stores data (metrics, logs, traces, etc).
- Grafana queries the data source to retrieve and visualize data in dashboards.
- Each data source has its own query language and configuration.

3. AUTHENTICATION AND TLS



- Most data sources support different authentication methods:
 - No authentication (internal networks)
 - Username and password
 - API tokens
 - AWS IAM roles (for CloudWatch)
 - TLS/SSL for encrypted connections
 - Always use HTTPS in production.

4. DATA-SOURCE HEALTH CHECKS



- Grafana provides a "Save & test" button to check the connection.
- It verifies:
 - Network connectivity
 - Authentication
 - Correct URL and port
 - Query capability (depends on source)
- A successful test confirms Grafana can reach and query the source.

2. COMMON PRODUCTION DATA SOURCES



Prometheus

- Metrics
- Uses PromQL



Loki

- Logs
- Uses LogQL



Tempo

- Traces
- Uses TraceQL



Elasticsearch

- Logs and search
- Uses Lucene Query DSL



CloudWatch

- Metrics, logs, alarms
- Used in AWS environments



PostgreSQL

- SQL data
- Useful for custom data and reporting



Other Supported Sources

- InfluxDB, MySQL, MSSQL
- OpenSearch
- Graphite, Jaeger, etc.
- Check Grafana documentation for full list

5. COMMON CONNECTION FAILURES



- Cannot reach host (network/firewall issue)
- Connection refused (wrong port or service not running)
- Authentication failed (wrong credentials/API key)
- TLS errors (invalid certificate or using HTTP instead of HTTPS)
- Timeout (service is slow or unreachable)
- No data returned (query or permissions issue)
- Wrong URL (missing path or incorrect endpoint)

6. TROUBLESHOOTING: GRAFANA CANNOT REACH PROMETHEUS



- 1 Check the Prometheus URL (e.g. <http://prometheus:9090>)
- 2 Verify Prometheus is running: `curl http://<prometheus-ip>:9090/-/healthy`
- 3 Check network connectivity (firewall, security group, DNS)
- 4 Ensure the correct port is open (default 9090)
- 5 Verify authentication settings (if enabled)
- 6 Check Grafana logs: `sudo journalctl -u grafana-server -f`
- 7 Test the query directly in Prometheus UI
- 8 Save & test the data source again in Grafana

7. PRACTICAL TIPS



- Use clear and consistent names for data sources
- Use HTTPS in production
- Restrict access using firewall/security groups
- Test connectivity from the Grafana server
- Use service discovery or DNS names
- Monitor data-source health regularly

8. KEY TAKEAWAY

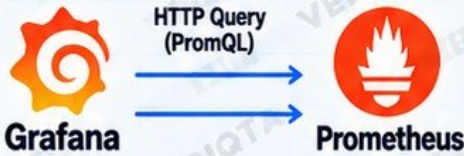


Grafana does not store your metrics, logs or traces. It queries data from external sources. A correctly configured and reachable data source is essential for meaningful dashboards.

Prometheus + Grafana in Production

From raw metrics to actionable insights

1. HOW GRAFANA QUERIES PROMETHEUS



- Grafana sends PromQL queries to Prometheus via its HTTP API.
- Prometheus returns time series data (JSON).
- Grafana visualizes the data in panels.

2. PROMQL INTRODUCTION

- PromQL is the query language for Prometheus.
- It is used to select, filter, aggregate and transform time series data.
- Basic syntax:

```
metric_name{label="value"}
```

- Example:

```
http_requests_total{job="api"}
```

3. INSTANT vs RANGE QUERIES

Instant Query

- Returns the latest value at a single point in time.
- Used for gauges (e.g. CPU, memory).

- Example: `up{job="api"}`

Range Query

- Returns a range of data over time.
- Used for rate, increase, avg, etc.

- Example: `rate(http_requests_total[5m])`

4. LABELS

- Labels are key-value pairs that identify time series (e.g. job, instance, environment).
- Used for filtering and grouping data.
- Example metric with labels:

```
http_requests_total{
  job="api",
  instance="10.0.0.1:8080",
  env="prod"
}
```

- Common labels: job, instance, namespace, pod, service, env, region.

5. RATE CALCULATIONS

- Use `rate()` for per-second averages of counter metrics.
- Use `increase()` for total increase over a time range.

- Example:

```
rate(http_requests_total[5m])
```

Common use cases:

- Request rate
- Error rate
- Traffic per service



6. QUERY INTERVALS

- Grafana uses a query interval to fetch data points.
- Range queries require a time window (e.g. [5m], [1h]).
- Shorter intervals give more detail but higher load on Prometheus.

- Example:

```
sum(rate(http_requests_total[5m]))
```

7. \$__RATE_INTERVAL

- `$__rate_interval` is a Grafana variable that automatically adjusts the rate interval based on the selected time range.
- Helps avoid PromQL errors and gives accurate results.

- Example:

```
rate(http_requests_total[$__rate_interval])
```

8. COMMON METRICS TO MONITOR

 CPU `rate(node_cpu_seconds_total[5m])`

 Memory `node_memory_usage_bytes`

 Request Rate `rate(http_requests_total[5m])`

 Latency `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))`

 Errors `rate(http_requests_total{status=~"5.."}[5m])`

9. WHY BADLY DESIGNED PROMQL CAN BE MISLEADING

- Using the wrong metric (e.g. gauge instead of counter).
- Incorrect or missing labels.
- Wrong rate interval (too short or too long).
- Not aggregating across instances (e.g. missing `sum()`).
- High cardinality queries (too many time series).
- These can lead to inaccurate dashboards, false alerts and wrong decisions.

10. KEY TAKEAWAY



Good PromQL queries are accurate, efficient and well designed. Always understand your metrics, labels and time ranges before building dashboards.

Metrics
Today
Better Systems
Tomorrow



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

"Good Metrics
Lead to Better Decisions"

Variables and Dynamic Dashboards

Make your dashboards flexible, reusable and scalable

1. DASHBOARD VARIABLES

- Dashboard variables allow you to make dashboards dynamic and reusable.
- They let you filter data without editing queries.
- Useful for different services, environments, clusters, etc.



Service

Namespace

Instance

2. QUERY VARIABLES

- Query variables get their values from a data source (e.g. Prometheus).
- They are updated automatically based on your data.
- Example (Prometheus):
`label_values(up, job)`
- Example (Kubernetes):
`label_values(kube_pod_info, namespace)`
- Use query variables for dynamic and real-time values.



3. CUSTOM VARIABLES

- Custom variables use a static list of values that you define manually.
- Useful for non-dynamic values like environments or regions.
- Example:
`prod, staging, dev`
- You can also set a default value.



4. DATA-SOURCE VARIABLES

- Used to select a data source dynamically in a dashboard.
- Useful when you have multiple Prometheus, Loki or other data sources.
- Example: Let users switch between different Prometheus instances.



Data source

5. MULTI-VALUE VARIABLES

- Allow selection of multiple values at the same time.
- Useful for comparing multiple services, namespaces, or instances.
- Example:
`label_values(up, job)`
- Enable "Multi-value" and "Include All option".



6. COMMON VARIABLE EXAMPLES

- `$service` → application or job name
- `$namespace` → Kubernetes namespace
- `$instance` → server or pod instance
- `$env` → environment (prod, staging, dev)
- `$region` → cloud region
- `$cluster` → Kubernetes cluster
- `$datasource` → data source name



7. REPEATING PANELS

- Repeating panels automatically create multiple panels based on a variable value.
- Useful for showing the same metric for multiple services or instances.
- Example: Repeat a panel for each `$service` value.



8. BUILDING ONE DASHBOARD FOR MANY SERVICES

- Use variables and repeating panels to monitor multiple services in a single dashboard.
- This makes it easier to manage and maintain.
- Example setup:
 - Create a `$service` variable
 - Use it in your queries
 - Enable repeating panels
 - Add common panels (CPU, memory, requests, errors, etc.)
 - Share one dashboard with all services



9. COMMON MISTAKE



Creating separate dashboards for every server or service.

This leads to:

- Harder maintenance
- Inconsistent dashboards
- More work when adding new services
- Difficult to get a global view



Use variables and repeating panels instead. Build once, use for many.

10. KEY TAKEAWAY



- Variables make your dashboards flexible, scalable and easy to maintain.
- Use query variables, multi-value variables and repeating panels to monitor many services in one dashboard.
- Avoid creating separate dashboards for every server.
- A well-designed dynamic dashboard saves time and gives better visibility across your entire infrastructure.

Same Dashboard
More Visibility
Less Work

Observability
Made Simple
with Grafana



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

The Four Golden Signals Dashboard

From metrics to meaningful insights in production

1. THE FOUR GOLDEN SIGNALS

The four golden signals give a simple but powerful view of your system's health.



Latency

How long requests take.



Traffic

How many requests you receive.



Errors

How many requests fail.



Saturation

How full your resources are (CPU, memory, disk, etc.).

2. SELECTING MEANINGFUL METRICS

- Choose metrics that reflect user experience and system health.
- Use RED (Rate, Errors, Duration) for services.
- Use USE (Utilization, Saturation, Errors) for infrastructure.
- Ensure metrics are reliable, accurate and actionable.
- Avoid too many metrics. Keep it simple.

"Not every metric is useful. Choose the ones that help you detect real problems."



3. BUILDING PANELS FOR EACH SIGNAL

- Create a panel for each golden signal in Grafana.
- Use clear titles, units and labels.
- Example metrics (Prometheus):

Latency

```
histogram_quantile(0.95,
rate(http_request_duration
_seconds_bucket[5m]))
```

Traffic

```
rate(http_requests_total[5m])
```

Errors

```
rate(http_requests_total{
status=~"5.."}[5m])
```

Saturation

```
node_cpu_utilization_seconds_total
node_memory_usage_bytes
```

4. HEALTHY vs UNHEALTHY BEHAVIOR

Signal	Healthy Behavior	Unhealthy Behavior
 Latency	Low and stable (e.g. < 200ms)	High or increasing (e.g. > 1s)
 Traffic	Stable or expected growth	Unexpected spike or sudden drop
 Errors	Very low (e.g. < 1%)	High or increasing error rate
 Saturation	Below capacity (e.g. < 70%)	High or near limit (e.g. > 90%)

5. TURNING TELEMETRY INTO AN OPERATIONAL STORY

- Dashboards should tell a story, not just show graphs.
- Look for patterns and correlations between signals.
- Example story:

- Traffic increases
- Latency starts to rise
- Errors begin to appear
- CPU/memory becomes saturated
- Identify the root cause and take action



"Good dashboards help you see the story before your users feel the pain."

6. PRODUCTION CONNECTION

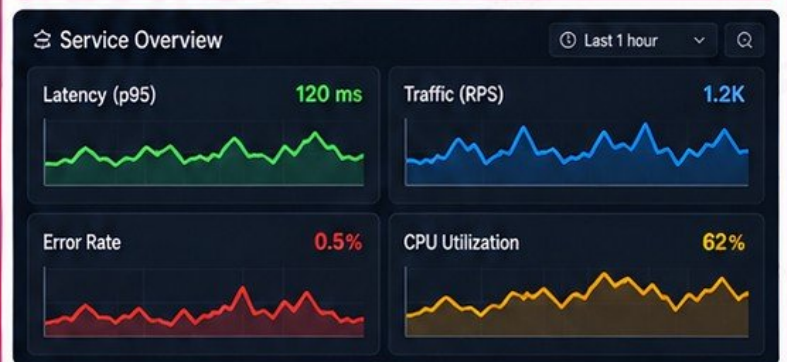


- The four golden signals help you detect degradation before total failure.
- They give early warning signs so you can investigate and fix issues proactively.
- Use alerts on key thresholds.
- Combine with logs and traces for deeper analysis.



Detect issues early. Fix problems before your users do.

7. EXAMPLE DASHBOARD LAYOUT



Kubernetes Monitoring with Grafana

From cluster metrics to actionable insights

1. CLUSTER vs WORKLOAD VISIBILITY

Cluster Visibility

- Overall health of the cluster.
- Nodes, control plane, system components.
- Helps you know if the cluster is healthy and has resources.

Workload Visibility

- Focuses on your applications.
- Pods, deployments, namespaces.
- Helps you understand how your services are performing.

You need both. A healthy cluster does not always mean healthy applications.



2. KEY KUBERNETES OBJECTS TO MONITOR



Nodes

- CPU, memory, disk, network.



Namespaces

- Resource usage and isolation.



Pods

- Resource usage, restarts, status.



Deployments

- Replica count, availability.

3. IMPORTANT METRICS



CPU

- Usage
- Requests vs limits



Memory

- Usage
- Requests vs limits



Restarts

- Container restart count



Pod status

- Running, Pending, Failed, Unknown

4. REQUESTS AND LIMITS

- Requests: the amount of resources guaranteed to a container.
- Limits: the maximum amount of resources a container can use.
- Monitor both to:
 - Detect resource pressure
 - Find overprovisioned or underprovisioned pods
 - Avoid application failures



"Right-sizing your requests and limits keeps your cluster stable and cost efficient."

5. KEY COMPONENTS



kube-state-metrics

- Exposes Kubernetes object metrics (pods, deployments, nodes, etc.).
- Used by Prometheus.



Prometheus

- Scrapes metrics from Kubernetes and kube-state-metrics.
- Stores time series data.
- Grafana queries Prometheus to visualize the metrics.

6. EXAMPLE FLOW



Kubernetes

Exporters

Prometheus

Grafana

(kube-state-metrics and others)

Kubernetes → Exporters → Prometheus → Grafana

Exporters collect metrics, Prometheus stores them, and Grafana visualizes them.

7. TROUBLESHOOTING CRASHLOOPBACKOFF



- CrashLoopBackOff means a container keeps failing and restarting.
- Use Grafana dashboards to investigate:

- Check pod restart count.
- Look at container logs (via Loki or kubectl).
- Review CPU and memory usage.
- Check events and error messages.
- Identify the root cause (config error, missing dependency, OOM, etc.).

"Dashboards give you the evidence. Logs and events tell you the story."

8. USEFUL DASHBOARD PANELS

Node CPU Usage

23%



Pod Memory Usage

512 MiB



Pod Restarts

7



Pod Status



Running	18
Pending	2
Failed	1
Unknown	0

9. KEY TAKEAWAY



- Monitor both cluster and workloads.
- Use kube-state-metrics and Prometheus.
- Track CPU, memory, restarts and pod status.
- Use dashboards to detect and troubleshoot issues early.
- Turn metrics into action before users are affected.

Observe
Troubleshoot
Improve
Keep Your
Services Running

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



Logs with Grafana Loki

From symptoms to root cause with logs

1. METRICS vs LOGS

Metrics



- Tell you something is wrong
- Show trends and patterns
- Example: High error rate, high CPU, high latency

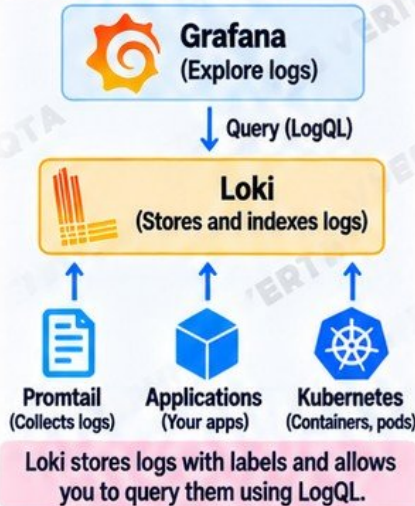
Logs



- Help explain what happened
- Show detailed events
- Example: Error messages, stack traces, application logs

*"Metrics alert you.
Logs give you the details."*

2. LOKI ARCHITECTURE (AT JUNIOR LEVEL)



3. LOG STREAMS

- A log stream is a set of log entries that share the same labels.
- Each unique combination of labels creates a separate stream.
- Example labels:

```
{app="api", namespace="prod", container="api"}
```

4. LABELS

- Labels add context to your logs.
- Used for filtering and searching.
- Good labels: app, namespace, container, environment.
- Avoid high-cardinality values (e.g. user_id, request_id, IP address).

5. LOGQL (LOKI QUERY LANGUAGE)

- LogQL is similar to PromQL but designed for logs.
- You can filter, search and parse logs.
- Basic syntax:

```
{label="value"} |= "error"
```

- Examples:

```
{app="api"} |= "error"
```

```
{namespace="prod"} |= "failed"
```

```
{app="web"} |~ "exception|error"
```

6. SEARCHING APPLICATION LOGS

- Use the Explore tab in Grafana.
- Select Loki as the data source.
- Write a LogQL query to search your logs.
- Example: find error logs for a service.

```
{app="api"} |= "error"
```



7. FILTERING ERRORS

- Filter logs using keywords, levels or patterns.
- Examples:

```
{app="api"} |= "ERROR"
```

```
{app="api"} |= "exception"
```

```
{app="worker"} |~ "fail|error|panic"
```

- Use time range to narrow the search.
- Combine with labels for better results.

8. FROM METRIC SPIKE TO RELATED LOGS

1. See a metric spike in Grafana (e.g. high error rate)
2. Switch to Loki and search for logs in the same time range
3. Find the related error messages and root cause



*Metrics show you there is a problem.
Logs help you understand why.*

9. COMMON MISTAKE



Using high-cardinality values as Loki labels.

- Examples of bad labels:

```
{user_id="12345"}  
{request_id="abc-123"}
```

- This creates too many log streams, increases storage and makes queries slow.
- Use labels with low cardinality (e.g. app, namespace, environment).

10. KEY TAKEAWAY



- Metrics tell you something is wrong.
- Logs explain what happened.
- Use Loki with meaningful labels.
- Learn basic LogQL for searching and filtering logs.
- Move from metric spikes to related logs to find the root cause.
- Avoid high-cardinality labels.
- Good logging practices make troubleshooting faster and easier.

Distributed Tracing with Grafana Tempo

See the full journey, find the real cause

1. WHY METRICS AND LOGS MAY NOT REVEAL REQUEST PATHS

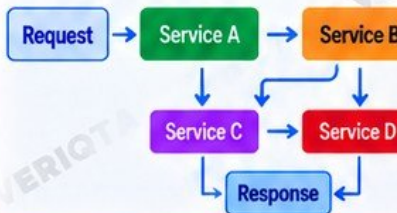
- Metrics tell you there is a problem (e.g. high latency).
- Logs show errors and events.
- But they don't show the full request journey across multiple services.
- In a distributed system, a single request passes through many services, and the problem may be in a downstream service that metrics and logs alone cannot easily identify.



Metrics and logs tell you something is wrong. Traces show you where and why.

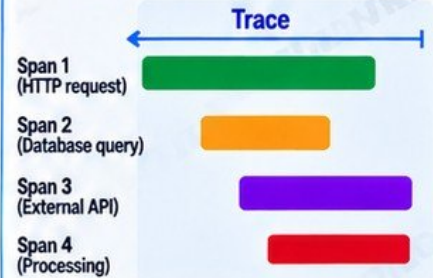
2. WHAT A TRACE REPRESENTS

- A trace represents the complete journey of a single request as it moves through multiple services.
- It shows how long each step takes and where time is spent.
- Helps you identify bottlenecks, dependencies and failures.



3. TRACES AND SPANS

- A trace is the full request journey.
- A span is a single unit of work within a trace (e.g. an API call, database query, external request).
- A trace contains multiple spans.



4. TRACE IDS

- Every trace has a unique Trace ID.
- All spans within the same request share the same Trace ID.
- You can use the Trace ID to find related logs and metrics.



Example Trace ID:

4f9e2c7b3d8a1e6f9c0d2b7a4e8c1234

5. SERVICE RELATIONSHIPS

- Traces show how services call each other.
- Helps you understand dependencies between services.
- You can see which service is slow or failing and how it affects the overall request.



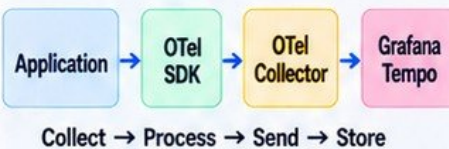
6. TEMPO

- Grafana Tempo is an open-source distributed tracing backend.
- It stores traces in a cost-effective way.
- Integrates natively with Grafana.
- Designed for high-scale, long-term trace storage.
- Works with OpenTelemetry, and other tracing tools.



7. OPENTELEMETRY CONNECTION

- OpenTelemetry (OTel) is the standard for collecting and sending traces.
- Instrument your applications with OTel SDKs (Python, Go, Java, etc.).
- Send traces to Grafana Tempo via the OTel Collector.



Collect → Process → Send → Store

8. FINDING SLOW REQUESTS

- Use Grafana Explore with Tempo.
- Search by Trace ID, service name or duration.
- Look for traces with high latency.
- Identify which span is taking the most time.
- Check tags, logs and service dependencies.



Trace ID	Duration	Services	Start Time
4f9e2c7b...	2.4s	api, db, payment	10:24:18
8a1d6e3c...	850ms	api, user, db	10:22:41
c7b2f9a1...	120ms	api, cache	10:21:03

9. MOVING FROM METRICS → LOGS → TRACES

- Metrics** Detect a problem (e.g. high latency)
- Logs** Find error messages and events
- Traces** See the full request journey and identify the root cause

"Metrics show the symptom. Logs give you clues. Traces show the full story."

10. REAL WORLD EXAMPLE: LOCATING A SLOW DOWNSTREAM SERVICE

- You notice high latency on the /checkout endpoint (metrics).
- You check logs and see no errors.
- You search traces for slow requests and find the Payment Service span takes 3 seconds.
- You investigate the Payment Service and find it is calling a third-party API that is slow.
- You fix the issue (e.g. add a timeout, retry or cache) and latency returns to normal.



Traces help you find the real cause, even when logs look normal.

Observe
Understand
Troubleshoot
Solve
Build Better

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta



Grafana Alerting in Production

From monitoring to action

1. DASHBOARD vs ALERT

Dashboard



- Shows you data visually.
- You check it manually.
- Helps you explore, understand and troubleshoot.
- Good dashboards do not replace alerts.

Alert



- Monitors data continuously.
- Notifies you when something is wrong.
- Helps you react faster.
- Based on alert rules, not just visuals.

"Dashboards help you see the present. Alerts help you catch the future."

2. ALERT RULES

- An alert rule defines when an alert should fire based on query results.
- Each rule has:

- 1 Queries (what data to evaluate)
- 2 Expressions (how to process the data)
- 3 Thresholds (when to trigger)
- 4 Evaluation intervals (how often to check)
- 5 Pending and firing states (to avoid false alerts)



"Good alert rules detect real problems, not just metric changes."

3. QUERIES, EXPRESSIONS AND THRESHOLDS

Query



- Get data from a data source (e.g. Prometheus, Loki).
- Example: CPU usage over time.

Expression



- Process the data (e.g. avg, max, rate, reduce).
- Example: average CPU over 5m.

Threshold



- Define the condition for alerting.
- Example: value > 80.

You can use multiple queries and expressions to create powerful alert conditions.

4. EVALUATION INTERVALS

- How often Grafana evaluates the alert rule.
- Common values: 10s, 30s, 1m, 5m.
- Short intervals detect issues faster but use more resources.
- Choose a value that matches your system and metric.



Example:

Evaluate every 1m, look at 5m of data.

5. PENDING AND FIRING STATES

- Pending:** condition is true, but the wait time is not yet over.
- Firing:** condition is true and the alert is officially active.
- Helps prevent alerts from firing due to short spikes.

Normal

Pending
(e.g. 2m)

Firing
(Alert!)

6. CONTACT POINTS

- Where Grafana sends notifications when an alert fires.
- Common contact points:



Slack



Email



Teams



PagerDuty



Webhook



etc.

7. NOTIFICATION POLICIES

- Define how alerts are routed to contact points.
- You can set rules based on:
 - Alert labels (e.g. severity, team)
 - Folder
 - Silences (mute alerts temporarily)
 - Grouping (combine multiple alerts)



"Send the right alert to the right team at the right time."

8. LABELS AND ANNOTATIONS

- Labels:** key-value pairs used for routing, grouping and filtering.
- Annotations:** additional information shown in notifications (e.g. description, runbook link).

Example:

Labels:

severity = critical
service = api
environment = production

Annotations:

summary = High CPU usage
description = CPU > 80% for 5m
runbook = <https://runbooks.com/cpu>



11. KEY TAKEAWAY

- Alerting turns your monitoring into action.
- Use meaningful conditions, not simple thresholds.
- Configure notification routing correctly.
- Provide clear labels and annotations.
- Avoid noisy alerts and focus on real problems.
- A well-designed alert helps you fix issues before users are affected.

10. WHY "CPU > 80%" IS OFTEN A POOR ALERT BY ITSELF



- Does not consider context (is it normal for this workload?)
- Can create too many false alerts.
- Does not tell you the user impact.
- Should be combined with other signals (e.g. sustained for 5m, high load, increased latency).
- Use better conditions, for example:

`avg(cpu_usage) > 80 for 5m AND request_rate is high`

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Designing Alerts That Engineers Trust

Less noise. More action. Happier teams.

1. SYMPTOMS vs CAUSES

- Symptoms tell you something is wrong (e.g. high CPU).
- Causes tell you why it is wrong (e.g. memory leak, high traffic, dependency failure).
- Good alerts help you investigate the cause, not just show a symptom.

Symptom
High CPU



Possible Causes

- High traffic
- Memory leak
- Stuck process

"Alert on useful symptoms, and give context to find the real cause."

2. ACTIONABLE ALERTS

- An alert should tell you what to do, not just that something is wrong.
- Include clear context, thresholds, and runbook links.
- Avoid vague alerts.

Bad Alert:
CPU > 80%



Good Alert:
CPU > 80% for 5 minutes
on prod-web-1

Runbook: <https://runbooks.com/cpu>



"An alert without a next step creates confusion."

3. SEVERITY

- Use severity levels to show impact and urgency.
- Common levels:
 -  **Critical** – service is down or major user impact.
 -  **Warning** – potential issue or degraded performance.
 -  **Info** – for visibility, not immediate action.



"Severity helps the right people respond at the right time."

4. WARNING vs CRITICAL

- **Warning:** something is abnormal, but the service is still working.
- **Critical:** service is unavailable or user experience is heavily impacted.
- Use different thresholds and notification channels.

Severity	Example	Action
Warning	Error rate > 2% for 5m	Investigate
Critical	Error rate > 5% for 5m	Immediate action

5. ALERT DURATION

- Use a time window to avoid false alerts.
- Example: trigger only if condition is true for 5 minutes.
- Short spikes are often normal.



Example:
CPU > 80% for 5m

Prevents alerts from short, temporary spikes.

6. NOISE REDUCTION

- Alert on meaningful conditions.
- Use proper thresholds and durations.
- Group related alerts.
- Suppress known, non-actionable alerts (e.g. during maintenance).
- Review and tune alerts regularly.



"Less noise means faster response to real problems."

7. FLAPPING

- Flapping happens when an alert keeps firing and resolving repeatedly due to unstable values.
- Causes: thresholds too close to normal values, short evaluation intervals, or highly variable metrics.
- Fix: use longer durations, better thresholds, and smoothing (e.g. avg).



Avoid alerts that flap.

8. MISSING DATA

- Missing data can mean:
 - the target is down
 - the exporter is failing
 - network issues
 - or a real problem
- Decide how to handle missing data (alert or ignore).
- Use the "No data" and "Error" settings in Grafana.



"No data can be a problem. Treat it intentionally."

9. ERROR-BUDGET THINKING

- Combine alerting with SLIs, SLOs and error budgets.
- Alert when you are at risk of breaching your SLO.
- Helps you focus on user impact, not just infrastructure metrics.



"Alert on what matters to your users."

10. RUNBOOK LINKS

- Always include a runbook link in your alert.
- The runbook should explain:
 - what the alert means
 - possible causes
 - steps to investigate
 - how to fix it



Runbook:
<https://runbooks.com/service-error>

11. OWNERSHIP

- Every alert should have a clear owner (team or person).
- Use labels to define ownership (e.g. team=payments).
- Ensure alerts go to the right people.



12. AVOIDING ALERT FATIGUE

- Too many alerts cause:
 - slower response times
 - important alerts being ignored
 - stress and burnout
- Regularly review, remove or tune low-value alerts.



"A smaller number of high-quality alerts is more effective than hundreds of noisy alerts."

13. JUNIOR ENGINEER TIP



Every alert should answer:
"What should I do?"

- Can I understand what is wrong?
- Do I know who owns it?
- Is there a runbook?
- Do I know the next step?
- If the answer is "no", improve the alert.

*Good Alerts
Build Reliable
Systems*

*Better
Engineers
Brighter
Tomorrows*



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Users, Teams, RBAC, and Security

Keep your Grafana environment secure and organized

1. USERS

- A user is an individual who logs in to Grafana.
- Each user has a unique identity and permissions.
- Users can be local (Grafana DB) or managed via an external authentication system (e.g. SSO).
- Best practice: use SSO instead of local users for teams.



"One user, one identity, proper access."

2. ORGANIZATIONS

- Grafana supports multi-tenancy using organizations.
- Each organization has its own dashboards, data sources, users and teams.
- A user can be a member of multiple organizations.



Use organizations to separate environments (e.g. dev, staging, production) or different teams.

3. TEAMS

- Teams help you group users (e.g. DevOps, SRE, Backend).
- You can assign permissions to teams instead of individual users.
- Makes access management easier and more scalable.



"Manage access by team, not by person."

4. ROLES

- Grafana has built-in roles:

Viewer

Can view dashboards (read-only).

Editor

Can create and edit dashboards.

Admin

Full access (users, data sources, settings, etc).

"Give the minimum role needed to do the job."

5. ROLE-BASED ACCESS CONTROL (RBAC)

- RBAC controls what users or teams can do in Grafana.
- Permissions can be assigned at the organization, folder or resource level (e.g. dashboards, data sources).
- Use RBAC to enforce the principle of least privilege.



"Right person. Right access. Right level."

6. AUTHENTICATION

- Grafana supports multiple authentication methods:
- Username and password (local)
- LDAP / Active Directory
- OAuth (Google, GitHub, etc.)
- SAML (for enterprise SSO)



Use a strong authentication method (SSO) for production environments.

7. SINGLE SIGN-ON (SSO)

- SSO allows users to log in with their company identity (e.g. Google, Azure AD, Okta).
- Simplifies user management.
- More secure than individual passwords.



Google



Azure AD



Okta

"One login. More security. Less management."

8. SERVICE ACCOUNTS

- Service accounts are non-human accounts used by applications, automation or CI/CD.
- Used with API tokens.
- Give only the permissions needed.



Example use cases:

- Automated dashboard provisioning
- CI/CD pipelines
- External tools and scripts

"Service accounts are for machines, not people."

9. API TOKENS

- API tokens allow programmatic access to Grafana.
- Used by scripts, automation tools and external systems.
- You can create tokens with specific scopes and expiration.



Never share API tokens publicly. Use environment variables or a secrets manager.

10. SECRETS

- Store sensitive information securely (e.g. API tokens, passwords, certificates).
- Use a secrets manager (e.g. HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets).
- Do not hardcode secrets in configuration files or scripts.



Keep secrets out of code and out of Git.

11. TLS

- Use TLS (HTTPS) to encrypt traffic between users and Grafana.
- Prevents attackers from sniffing credentials or tokens.
- Use a valid SSL/TLS certificate (e.g. from Let's Encrypt or your internal CA).



Always use HTTPS in production."

12. LEAST PRIVILEGE

- Give users, teams and service accounts only the access they need.
- Avoid using the Admin role unless necessary.
- Regularly review and remove unnecessary access.



Minimal access. Maximum security."

13. PROTECTING ADMINISTRATIVE ACCESS

- Restrict admin access to trusted users only.
- Use SSO and multi-factor authentication (MFA).
- Monitor admin activity.
- Avoid shared accounts.
- Use strong passwords and rotate credentials.



Admin access should be limited, monitored and protected.

Better
Engineers
Brighter
Tomorrows



Security Reminder:

never expose Grafana anonymously without understanding the risk.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Dashboards as Code and Provisioning

Automate. Version. Reproduce. Scale.

1. WHY MANUAL CLICKS DO NOT SCALE

- Clicking in the UI works for small setups but does not scale.
- Manual changes are hard to track.
- Different environments become inconsistent (dev, staging, prod).
- There is no easy way to review changes or revert mistakes.
- Teams cannot collaborate effectively.



"If you click everything manually, you will eventually create drift, confusion and outages."

2. DASHBOARD JSON

- Grafana dashboards are stored as JSON.
- The JSON defines panels, queries, variables, layout and settings.
- You can export a dashboard JSON from Grafana and store it in Git.
- You can edit JSON manually or use tools to generate it.

Example (partial dashboard JSON):

```
{
  "title": "API Overview",
  "panels": [
    {
      "type": "timeseries",
      "title": "Request Rate"
    }
  ]
}
```



3. DATA-SOURCE PROVISIONING

- Define data sources in configuration files instead of creating them manually in the UI.
- Supported via Grafana provisioning.
- Common data sources: Prometheus, Loki, Tempo, etc.

Example (datasource YAML):

```
apiVersion: 1

datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
```



4. DASHBOARD PROVISIONING

- Store dashboard JSON files in a folder and let Grafana load them automatically.
- Use provisioning configuration (YAML).
- You can provision multiple dashboards at once.

Example (dashboard provisioning YAML):

```
apiVersion: 1

providers:
- name: 'veriqta-dashboards'
  orgId: 1
  folder: 'Production'
  type: file
  options:
    path: /var/lib/grafana/dashboards
```



5. VERSION CONTROL

- Store dashboard JSON and provisioning files in Git.
- Track changes, review updates and roll back if needed.
- Use meaningful commit messages.



Example commit messages:

- Add API latency dashboard
- Update Prometheus datasource
- Fix panel query
- Change threshold value

"Version control gives you history, accountability and safe collaboration."

6. GIT AND TERRAFORM

- Use Git to store JSON and provisioning files.
- Use Terraform to manage Grafana resources (dashboards, data sources, folders, teams, etc.).
- Helps you treat Grafana as infrastructure as code.



Git
(Version control)



Terraform
(Infrastructure as code)

"Same tools. Same process. More consistent environments."

7. REPRODUCIBILITY

- Provisioning ensures you can recreate the same Grafana setup anywhere (dev, staging, prod).
- No manual steps.
- Reduces human error.
- Makes disaster recovery easier.



"Reproducible monitoring builds reliable systems."

8. ENVIRONMENT PROMOTION

- Use the same configuration files across environments.
- Promote changes from dev → staging → production.
- Keep environments consistent.

Dev → Staging → Production

"Same config. Different environments. Fewer surprises."

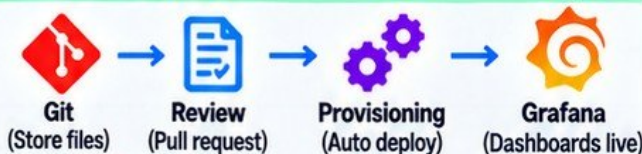
9. REVIEWABLE DASHBOARD CHANGES

- Changes are reviewed through pull requests.
- Team members can suggest improvements.
- Catches mistakes before they reach production.
- Provides a clear audit trail.



"Review before you deploy. Better dashboards, fewer problems."

10. COMPLETE FLOW



11. KEY TAKEAWAYS



- Do not rely on manual clicks.
- Use JSON, provisioning, Git and Terraform.
- Keep your environments consistent.
- Make changes reviewable.
- Automate your monitoring setup.
- Dashboards as code saves time and reduces risk.

Automate
Monitor
Scale
Succeed

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Grafana Performance and Scalability

Keep Grafana fast, reliable and ready to grow

1. WHAT HAPPENS WHEN GRAFANA BECOMES SLOW

- Dashboards take a long time to load.
- Queries timeout or fail.
- Users experience a poor experience and lose trust.
- Teams stop using Grafana.
- Alerts and troubleshooting become slower.



"A slow Grafana is a productivity killer. Fix performance before users stop relying on it."

2. EXPENSIVE QUERIES

- Queries that scan a lot of data are expensive.
- Using complex functions (rate, regex, joins, high cardinality) increases load.
- Unoptimized queries can overload your data source (e.g. Prometheus).

Example (expensive query):

```
rate(http_requests_total[24h])
by (pod)
```



Use specific time ranges, filters and proper aggregations.

3. LARGE TIME RANGES

- Querying long time ranges (e.g. 30d, 90d) returns a lot of data.
- Increases memory and CPU usage on Grafana and the data source.
- Use shorter ranges for dashboards (e.g. 1h, 6h, 24h) and long ranges only when needed.



"Only query what you need."

4. EXCESSIVE PANELS

- Too many panels on a single dashboard slows down loading time.
- Each panel runs one or more queries.
- Use only relevant panels.
- Consider splitting large dashboards into smaller, focused dashboards.



"Quality over quantity. Show what matters."

5. HIGH-CARDINALITY METRICS

- High cardinality means too many unique label values (e.g. user_id, session_id, pod, container).
- It increases storage, memory and query time.
- Avoid using high-cardinality labels in dashboards and alerts.
- Aggregate data when possible.

"High cardinality can make your queries and dashboards painfully slow."

6. QUERY CONCURRENCY

- Many users running queries at the same time can overload Grafana and the data source.
- Grafana has limits for concurrent queries.
- Tune concurrency settings based on your resources.



"Control concurrency to keep your system stable."

7. CACHING CONCEPTS

- Grafana can cache query results to improve performance.
- Caching reduces load on the data source.
- Use built-in caching or a proxy cache (e.g. query caching, CDN).
- Set appropriate cache TTL (time to live).



"Cache smartly. Fresh enough data, faster dashboards."

8. DATABASE CONSIDERATIONS

- Grafana stores users, dashboards, settings, alerts, etc. in a database (e.g. SQLite, MySQL, PostgreSQL).
- For production, use MySQL or PostgreSQL (not SQLite).
- Ensure proper sizing, backups and high availability.



"A reliable database keeps Grafana reliable."

9. HORIZONTAL SCALING

- Run multiple Grafana instances to handle more users and load.
- Use a load balancer (e.g. Nginx, HAProxy, cloud load balancer).
- All instances must use the same database and shared config.



"Scale out when one instance is not enough."

10. LOAD BALANCING

- Distribute traffic across multiple Grafana instances.
- Use health checks to remove unhealthy instances.
- Common options: Nginx, HAProxy, AWS ALB, Cloudflare.
- Ensure sticky sessions are not required (Grafana is stateless).



"A good load balancer keeps Grafana available and responsive."

11. SHARED DATABASE REQUIREMENTS

- All Grafana instances must use the same database.
- Keeps dashboards, users, alerts and settings in sync.
- Use a highly available database setup in production.
- Regular backups are essential.



"One database. Many instances. Always in sync."

12. TROUBLESHOOTING SLOW DASHBOARDS SYSTEMATICALLY

- 1 Check query time (use Query Inspector).
- 2 Look for expensive queries (high cardinality, large range).
- 3 Reduce the time range and number of panels.
- 4 Check data source performance (Prometheus, Loki, etc.).
- 5 Review Grafana logs for errors or slow queries.
- 6 Check server resources (CPU, memory, database).
- 7 Optimize and test again.



"Find the bottleneck. Fix it. Test it. Repeat."

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Grafana High Availability, Backup, and Recovery

Stay available. Protect your data. Be ready to recover.

1. WHY GRAFANA CAN BE CRITICAL INFRASTRUCTURE

- Teams rely on Grafana for monitoring, alerting and incident response.
- If Grafana is down, engineers lose visibility during incidents.
- Dashboards, alerts and configuration represent operational knowledge.
- In large organizations, Grafana supports multiple teams and business-critical services.



"Grafana may not run your applications, but when it's down, running your applications becomes much harder."

4. LOAD BALANCER

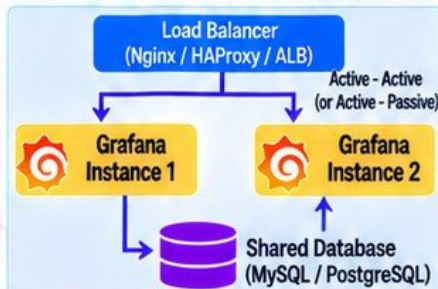
- Distributes traffic across Grafana instances.
- Performs health checks to remove unhealthy instances.
- Common options: Nginx, HAProxy, AWS ALB, Cloud Load Balancer.
- Configure session persistence only if needed.



"A load balancer keeps Grafana available and responsive."

2. HIGH-AVAILABILITY ARCHITECTURE

- Run multiple Grafana instances (at least 2).
- Use a load balancer (e.g. Nginx, HAProxy, AWS ALB).
- Use a shared database (e.g. MySQL, PostgreSQL).
- Store configuration and plugins persistently.
- Design for automatic recovery and minimal downtime.



5. SHARED DATABASE

- Store all Grafana data in a shared database (e.g. MySQL or PostgreSQL).
- Do not use SQLite for high availability (multi-instance).
- Ensure database high availability (backups, replication, failover).
- Monitor database performance and storage.



"The database is the heart of Grafana. Keep it highly available and backed up."

3. MULTIPLE GRAFANA INSTANCES

- Run two or more instances for high availability.
- Instances should be stateless.
- Use a shared database for users, dashboards, alerts and settings.
- Keep Grafana versions consistent across instances.
- Use health checks on the load balancer.



"Multiple instances remove single points of failure."

6. PERSISTENT CONFIGURATION

- Store configuration files (grafana.ini) on persistent storage or configuration management (e.g. Git, Ansible, Terraform).
- Keep the same configuration across all instances.
- Manage environment-specific settings securely (secrets, external config).



"Consistent configuration ensures predictable behavior across instances."

7. BACKING UP GRAFANA DATA

- Back up the Grafana database regularly (users, dashboards, alerts, settings).
- Store backups in a secure, offsite location (e.g. S3, Azure Blob, another region).
- Automate backups and monitor backup success.
- Test your backups periodically.



Example (PostgreSQL):

```
pg_dump -U grafana -Fc grafana > grafana-backup.dump
```

8. DASHBOARDS AS CODE

- Store dashboard JSON files in Git.
- Use provisioning to automatically deploy dashboards.
- Makes recovery and environment promotion easier.
- Treat dashboards like application code (review, version control, history).



"If it's important, keep it in Git."

9. PLUGINS

- Keep a list of installed plugins (plugin IDs and versions).
- Backup plugin configuration (if any).
- Ensure plugins are available during recovery (internet access or internal repository).



"Plugins extend Grafana. Document them so you can restore them."

10. RECOVERY TESTING

- A backup is not proven until you have successfully restored it.
- Regularly test restoring the database, dashboards, configuration and plugins.
- Verify that alerts, data sources and users work after recovery.
- Document the recovery process.



"Test today, so you're ready tomorrow."

11. DISASTER RECOVERY CONSIDERATIONS

- Keep backups in a different region or cloud provider.
- Document RTO (Recovery Time Objective) and RPO (Recovery Point Objective).
- Automate infrastructure deployment (Terraform, CloudFormation, etc.).
- Have a clear runbook for recovery.



"Plan for the worst, so you can recover faster."

12. KEY TAKEAWAYS

- ✓ Grafana can be critical infrastructure.
- ✓ Use multiple instances and a load balancer.
- ✓ Use a shared, highly available database.
- ✓ Keep configuration persistent and consistent.
- ✓ Back up your data regularly.
- ✓ Store dashboards as code.
- ✓ Test your recovery process.
- ✓ A backup is not proven until restoration works.

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Troubleshooting Grafana Production Failures

Find the issue. Fix it. Keep your systems reliable.

1. TROUBLESHOOTING FLOW

Use a structured approach:



- Be methodical and avoid random changes.
- Collect evidence before making assumptions.
- Verify the fix and monitor for recurrence.

"A clear process turns a problem into a solution."

2. USEFUL EVIDENCE

- `systemctl status grafana-server`
- `journalctl -u grafana-server`
- Grafana logs (`/var/log/grafana/grafana.log`)
- Data-source health (Grafana UI)
- Browser console and network errors
- Backend telemetry (metrics, logs, traces)
- Server resources (CPU, memory, disk)
- Configuration files (`grafana.ini`, provisioning)

```

$ systemctl status grafana-server
$ journalctl -u grafana-server -f
$ tail -n 50 /var/log/grafana/grafana.log
  
```

"Evidence beats assumptions."

3. COMMON SCENARIOS (OVERVIEW)

- 1 Grafana will not start
- 2 Data source unavailable
- 3 "No data" panel
- 4 Dashboard suddenly empty
- 5 Prometheus query timeout
- 6 Loki logs missing
- 7 Authentication failure
- 8 Alert not firing
- 9 Notifications not delivered
- 10 Dashboard extremely slow
- 11 Plugin failure
- 12 Permission denied
- 13 Time-range mismatch

Same Problems.
Better Engineers.

"Different symptoms. Same process."

4. GRAFANA WILL NOT START

- Check service status and logs.
- Verify configuration (`grafana.ini`).
- Check database connectivity.
- Look for port conflicts (default 3000).
- Check permissions and disk space.

"Check logs. They always tell a story."

5. DATA SOURCE UNAVAILABLE

- Check data-source health in Grafana.
- Verify URL, credentials and network connectivity.
- Test from the Grafana server (`curl`, `ping`).
- Check firewall and DNS.

"If the data source is down, dashboards show no data."

6. "NO DATA" PANEL

- Check query and time range.
- Verify the data source is working.
- Check filters, labels and variables.
- Look for mismatched environment or namespace.

"No data often means a query, filter or time-range issue."

7. DASHBOARD SUDDENLY EMPTY

- Check folder permissions.
- Verify data sources and queries.
- Look for deleted or changed variables.
- Check if the time range is correct.
- Review recent changes (Git/provisioning).

"Dashboards don't disappear by themselves."

8. PROMETHEUS QUERY TIMEOUT

- Query is too heavy or complex.
- Time range is too large.
- High-cardinality labels.
- Prometheus is under high load.
- Check Prometheus logs and resources.

"Reduce time range, simplify the query, check Prometheus health."

9. LOKI LOGS MISSING

- Check Loki data source health.
- Verify labels and query syntax.
- Ensure logs are being shipped (e.g. `promtail`).
- Check time range and stream labels.

"Logs exist, but you may be querying the wrong stream."

10. AUTHENTICATION FAILURE

- Check user credentials.
- Verify LDAP/OAuth/SSO configuration.
- Check organization and team mapping.
- Review Grafana logs.

"Authentication issues are often config or permission related."

11. ALERT NOT FIRING

- Check alert rule query and condition.
- Verify evaluation interval.
- Check if the condition is actually met.
- Look at alert state (pending/firing).
- Review Grafana logs.

"Test alerts. Don't assume they work."

12. NOTIFICATIONS NOT DELIVERED

- Check contact point configuration.
- Verify integration (email, Slack, etc.).
- Check network connectivity.
- Look for errors in notification logs.
- Test with a sample alert.

"An alert is useless if nobody receives it."

13. DASHBOARD EXTREMELY SLOW

- Too many panels or queries.
- Large time ranges.
- Expensive queries or high-cardinality data.
- Browser performance issues.
- Check Grafana server resources.

"Optimize queries and reduce unnecessary panels."

14. PLUGIN FAILURE

- Check plugin installation and version.
- Look for errors in Grafana logs.
- Verify compatibility with Grafana version.
- Try disabling and re-enabling the plugin.

"Check logs and plugin versions."

15. PERMISSION DENIED

- Check user role and organization.
- Verify folder and dashboard permissions.
- Review team membership.
- Look for RBAC or LDAP group mapping issues.

"Right user. Right role. Right access."

16. TIME-RANGE MISMATCH

- Check the selected time range.
- Verify data actually exists in that period.
- Look for timezone differences.
- Be consistent across panels.

"The data is there. You might just be looking at the wrong time."

17. KEY TAKEAWAYS

- ✓ Follow a structured troubleshooting flow.
- ✓ Use logs, metrics and UI to gather evidence.
- ✓ Check configuration, data sources and permissions.
- ✓ Verify the fix and monitor for recurrence.
- ✓ Document common issues and solutions.
- ✓ Test alerts and notifications.
- ✓ Keep dashboards and queries optimized.
- ✓ A backup plan and recovery process are essential.



Production Awareness:
A backup is not proven until restoration works.

"Prepare. Test. Recover. That's real reliability."

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Production Project: Build a Complete Grafana Observability Environment

Bring it all together. Real tools. Real skills. Real-world scenarios.

1. PROJECT OVERVIEW

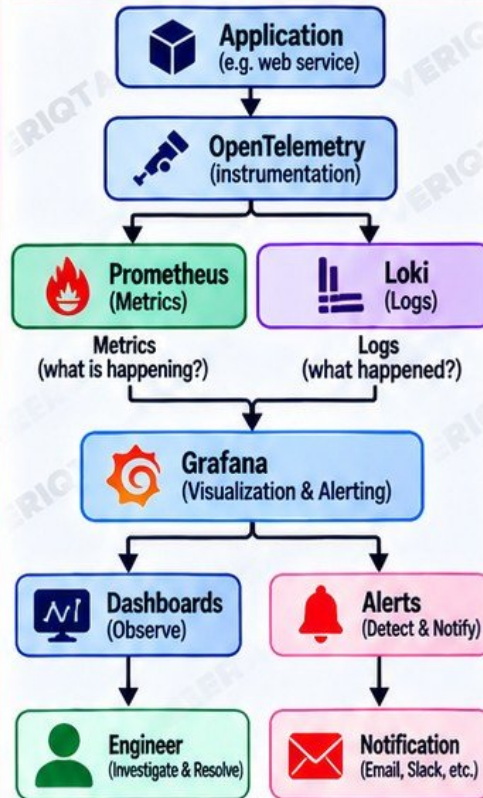
- Build a complete observability environment using industry-standard tools.
- Collect metrics, logs and traces from an application.
- Visualize everything in Grafana.
- Create useful dashboards and alerts.
- Secure, automate and make it production-ready.
- Investigate a real incident scenario.

*"From data to insight.
From insight to action."*

4. TOOLS USED

-  **Grafana**
Dashboards & alerting
-  **Prometheus**
Metrics collection
-  **Loki**
Log aggregation
-  **Tempo**
Distributed tracing
-  **OpenTelemetry**
Instrumentation
-  **Kubernetes**
Container orchestration

2. ARCHITECTURE DIAGRAM



3. WHAT THE LEARNER IMPLEMENTS

- ✓ Grafana (visualization & alerting)
- ✓ Prometheus metrics
- ✓ Loki logs
- ✓ Tempo traces
- ✓ OpenTelemetry (instrumentation)
- ✓ Kubernetes visibility
- ✓ Service dashboard
- ✓ Four Golden Signals (latency, traffic, errors, saturation)
- ✓ Variables (environment, service, etc.)
- ✓ Alert rules
- ✓ Contact points (email, Slack, etc.)
- ✓ Authentication and access controls
- ✓ Dashboard provisioning
- ✓ Git-based configuration
- ✓ Backup strategy
- ✓ Basic Grafana monitoring (self-monitoring)
- ✓ Failure investigation

*"Build it. Secure it. Monitor it.
Be ready for real incidents."*

5. SAMPLE DASHBOARD PANELS

- Request rate (RPS)
- Request latency (p95/p99)
- Error rate (4xx/5xx)
- CPU and memory usage
- Pod health (Kubernetes)
- Log errors (Loki)
- Trace latency (Tempo)



*"One dashboard.
A complete view."*

6. ALERT EXAMPLES

- High error rate (> 5%)
- High latency (p95 > threshold)
- Application down
- Kubernetes pod restarts
- High CPU or memory usage
- No data (missing metrics)



*"Alert on what matters.
Avoid noise."*

7. PROVISIONING AND AUTOMATION

- Store dashboards as JSON in Git
- Use provisioning for dashboards and data sources
- Manage configuration with Git
- Automate deployments (Terraform or Kubernetes manifests)



*"Infrastructure as code
for observability."*

8. FINAL INCIDENT EXERCISE

Introduce a slow application with elevated errors. As a junior engineer, you must:

- 1 Detect the problem in Grafana.
- 2 Identify the affected service.
- 3 Correlate metrics with logs and traces.
- 4 Determine the likely cause.
- 5 Apply the fix.
- 6 Verify recovery.



*"This is where theory
becomes real engineering."*

9. SUCCESS CRITERIA

- ✓ All components are running and connected
- ✓ Metrics, logs and traces are visible in Grafana
- ✓ Useful dashboards are created
- ✓ Alerts are configured and tested
- ✓ Access is secured
- ✓ Dashboards are provisioned from Git
- ✓ Backup and recovery plan is documented
- ✓ Incident is successfully investigated and resolved

*"You built a production-ready
observability environment."*

10. KEY TAKEAWAYS

- ✓ Observability is more powerful when metrics, logs and traces work together.
- ✓ Grafana is the central place to investigate and respond.
- ✓ Automation and Git keep your setup consistent and reliable.
- ✓ Real incidents will happen. Be prepared.



"Observe. Understand. Fix. Grow."

Better
Engineers
Brighter
Tomorrows



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta